

DOSSIER DE PROJET

pour le passage du titre professionnel

Développeur Web et Web Mobile

[ROZOTTE Lucas]

Projets présentés :

Astral Database — <https://github.com/Lucas259746/projet-hsr-bulma>

Cahier de recettes collaboratif — <https://github.com/Lucas259746/Cahier-de-recettes-collaboratif>

MarsAI (projet de groupe réalisé durant le stage, contribution frontend) —
<https://github.com/ivann-machado/marsAI>

Sommaire

Chapitre 1 — Astral Database (contribution individuelle, front-end)

Présentation du projet : Astral Database

CP1 — Installer et configurer son environnement de travail en fonction du projet web ou web mobile

CP2 — Maquetter des interfaces utilisateur web ou web mobile

CP3 — Réaliser des interfaces utilisateur statiques web ou web mobile

CP4 — Développer la partie dynamique des interfaces utilisateur web ou web mobile

Complément : la couche serveur que j'ai construite pour Astral Database

Sécurité et bonnes pratiques — Astral Database

Chapitre 2 — Cahier de recettes collaboratif (contribution individuelle, back-end)

Présentation du projet : Cahier de recettes collaboratif

Installer et configurer son environnement de travail (complément back-end)

CP6 (volet NoSQL) — Développer des composants d'accès aux données NoSQL

CP7 — Développer des composants métier côté serveur

CP8 — Documenter le déploiement d'une application dynamique web ou web mobile

Sécurité et bonnes pratiques — Cahier de recettes collaboratif

Chapitre 3 — MarsAI (projet de groupe, contribution frontend)

Conception collaborative : MCD, MLD et maquettes

Mes composants frontend

Ce que je retiens de cette expérience de groupe

Sécurité et bonnes pratiques — MarsAI

Chapitre 4 — Compétences transversales et conclusion

RGPD et données personnelles

Communiquer en français et en anglais

Mettre en oeuvre une démarche de résolution de problème

Apprendre en continu

Chapitre 1 — Astral Database (contribution individuelle, front-end)

Présentation du projet : Astral Database

J'ai développé Astral Database après le projet Mars Ai par curiosité pour les api, je me suis donc mis en tête de créer une vitrine de personnages, à partir de l'identifiant unique (UID) d'un compte, dans une interface pensée pour ce jeu plutôt que dans le format brut d'une réponse JSON.

J'ai voulu construire quelque chose qui ressemble, dans son ambiance visuelle, à l'interface du jeu lui-même : thème sombre, dorures, typographies Orbitron et Inter, cadres de rareté colorés selon que l'objet affiché est à trois, quatre ou cinq étoiles.

J'ai choisi de m'appuyer sur l'API publique Mihomo (api.mihomo.me), qui expose les données affichées par le jeu sur la vitrine d'un joueur (personnages, cônes de lumière, reliques, statistiques).

Plutôt que d'interroger cette API directement depuis le navigateur, j'ai construit deux ensembles distincts qui communiquent par une API REST interne : un frontend en React 19 avec le bundler Vite, stylé avec le framework CSS Bulma que j'ai complété par des surcharges personnalisées dans App.css, et un backend Node.js/Express que j'ai conçu comme un proxy applicatif entre mon frontend et l'API distante.

J'explique à la fin de cette partie pourquoi j'ai fait ce choix d'architecture plutôt que d'appeler l'API tierce directement depuis React.

Astral Database — architecture frontend / backend proxy

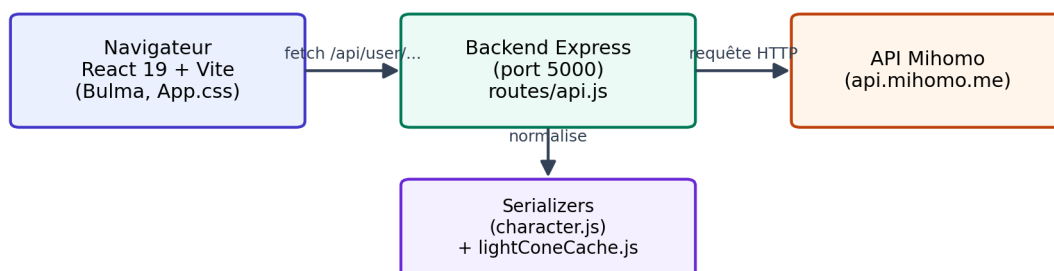


Figure — Architecture frontend / backend proxy d'Astral Database.

Ce projet m'a permis de travailler sur un problème que je n'avais pas vraiment pu développer : consommer une source de données externe que je ne contrôle pas, peu documentée, et dont le format change selon les cas (par exemple, certains personnages ont des compétences groupées en plusieurs formes, d'autres non).

J'ai dû construire une interface capable d'absorber ces variations sans se casser, tout en gardant un frontend lisible et une séparation claire entre la logique et l'affichage.

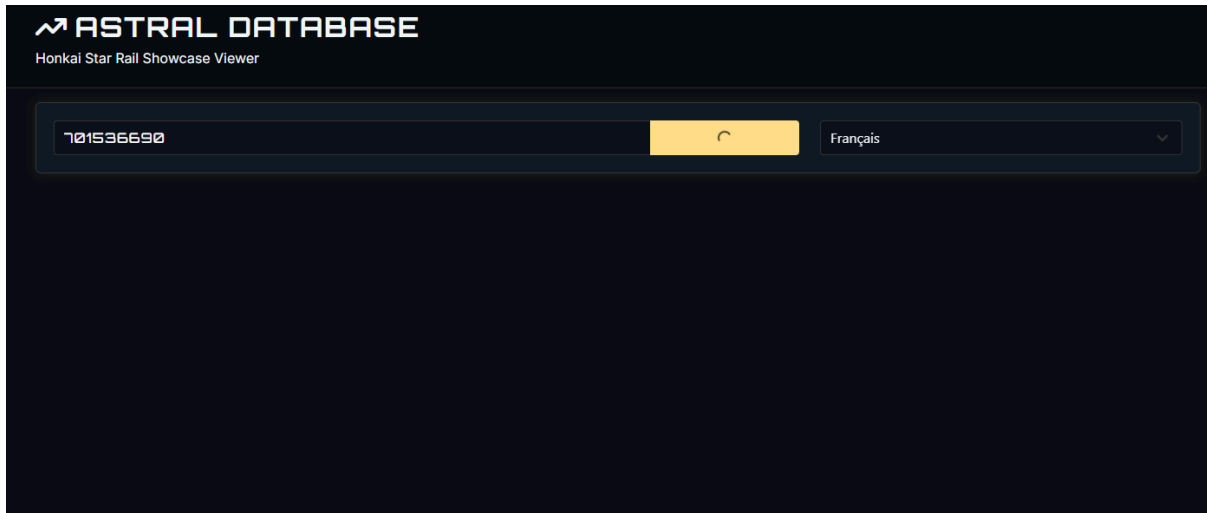
Voici comment j'ai organisé les fichiers de mon projet, pour que le jury puisse s'y retrouver rapidement si je dois naviguer dans mon code pendant l'entretien technique :

```
frontend/
  src/App.jsx, App.css      - mon composant racine, état global, thème
  src/components/characterComp/ - CharacterList, CharacterDetails, CharacterStats
  src/components/bottomSection/ - BottomSection, SkillCard, useBottomSection
  src/components/bottomSection/skillTreePanel/ - SkillTreePanel, useSkillTree
  src/components/lightConeComp/ - LightConeCard
  src/components/relicComp/    - RelicCard, RelicSection
  src/components/pathComp/     - pathLayouts.js + 9 fichiers de voies
  src/imageMap/               - imageMap.js + assets par personnage
backend/
  server.js                  - point d'entrée Express, middlewares, CORS
  routes/api.js              - routes /api/user/:userId, /raw
  config/mihomo.js           - mon client HTTP vers l'API Mihomo
  config/lightConeCache.js    - mon cache mémoire des descriptions de cônes
  utils/serializers/         - character.js, lightCone.js, relic.js, skill.js,
  sumStats.js, helpers.js
```

Ce que je retiens de ce choix d'architecture en deux parties (frontend + backend proxy), c'est qu'il m'a obligé à réfléchir très tôt à la forme des données que je voulais manipuler côté React, plutôt que de subir directement le format de l'API Mihomo dans mes composants.

Concrètement, mes composants React ne connaissent jamais la structure brute de l'API : ils ne manipulent que les objets propres que mes serializers leur préparent. Si l'API change un jour de format, je n'ai qu'un seul endroit à corriger — mes serializers — plutôt que de traquer le problème dans une dizaine de composants différents.

C'est une leçon d'architecture que je n'avais appliquée qu'en théorie avant ce projet, et que j'ai pu vérifier concrètement en cours de développement, quand j'ai justement dû ajuster mon code après avoir constaté que l'API renvoyait tantôt `max_level`, tantôt `maxLevel` selon les personnages.



Installer et configurer son environnement de travail en fonction du projet web ou web mobile

J'ai organisé mon projet en deux sous-projets Node.js distincts au sein d'un même dépôt : un dossier frontend piloté par Vite et un dossier backend piloté par Express. J'ai défini dans le `package.json` à la racine du frontend quatre scripts npm distincts, que j'orchestre avec le paquet `concurrently` :

```
"server": "nodemon server.js", // lance le backend Express avec rechargement auto
"client": "vite",               // lance le serveur de développement Vite
"dev": "concurrently \"npm run server\" \"npm run client\"",
"build": "vite build",
"lint": "eslint .",
"preview": "vite preview"
```

Quand je lance `npm run dev`, cela exécute simultanément mon serveur Express (avec `nodemon`, qui surveille les fichiers `.js` et relance le processus à chaque modification que je fais) et le serveur de développement Vite (qui recharge le frontend à la volée via le Hot Module Replacement).

J'ai mis en place cette configuration pour pouvoir travailler sur les deux couches de l'application en parallèle, sans avoir à relancer manuellement les processus après chaque modification — un gain de temps réel une fois que je développais activement les deux couches en même temps.

Côté backend, j'ai déclaré dans `server.js` le port d'écoute (5000) et initialisé les middlewares `express.json()` et `cors()`.

J'ai dû activer CORS explicitement car mon frontend (servi par Vite, sur le port 5173) et mon backend tournent sur deux origines différentes en développement ; sans ce middleware, le navigateur bloque les requêtes par sécurité.

J'ai ensuite délégué l'ensemble des routes préfixées par `/api` au routeur que j'ai défini dans `routes/api.js`, pour garder `server.js` court et centré sur la configuration plutôt que sur la logique métier.

J'ai séparé mes dépendances de production (`express`, `cors`, `react`, `react-dom`, `concurrently`, `nodemon`) de mes dépendances de développement (outillage Vite, ESLint, types TypeScript).

J'ai versionné mon projet avec Git dès le début, ce qui m'a permis de conserver l'historique de mes modifications et de revenir en arrière à plusieurs reprises quand une expérimentation ne fonctionnait pas comme prévu.

Point de vigilance que j'ai identifié en relisant mon propre code : j'ai codé en dur l'URL de mon API backend dans `App.jsx` (`http://localhost:5000/api/user/...`), ce qui fonctionne très bien en développement local mais m'empêcherait de déployer l'application en production sans modifier le code source.

Je sais qu'il me faudrait introduire une variable d'environnement (par exemple `VITE_API_URL`, lue via `import.meta.env`) avant toute mise en production,

Maquetter des interfaces utilisateur web ou web mobile

J'ai défini l'organisation visuelle de mon application (répartition en colonnes de la liste des personnages, de la fiche détail et du cône de lumière, puis en onglets pour les aptitudes, les reliques et l'arbre de compétences) directement par itération dans le code, en m'appuyant sur le système de grille du framework Bulma (classes `columns` / `column`), plutôt qu'en produisant une maquette au préalable dans un outil dédié comme Figma ou Penpot.

C'est une méthode de travail que j'ai adoptée par habitude sur ce projet personnel, mais je suis conscient qu'elle ne correspond pas à la démarche attendue par le référentiel, qui demande la production de maquettes formalisées ainsi qu'un schéma de l'enchaînement des interfaces avant le développement.

Je considère ce point comme une insuffisance de mon dossier que je dois corriger avant ma session d'examen : je prévois de reconstituer, a posteriori, une maquette basse ou moyenne fidélité de mon écran principal (recherche, liste de personnages, détail) ainsi qu'un schéma d'enchaînement simple (recherche → sélection d'un personnage → consultation des trois onglets), à l'aide d'un outil de maquettage gratuit comme Figma ou Penpot.

Le fait que mon application existe déjà et fonctionne me permet justement de documenter précisément la structure de navigation que j'ai construite, ce qui facilite cette reconstitution.

Je peux dès à présent décrire cette structure de navigation telle qu'elle existe dans mon code, pour servir de base à cette maquette rétrospective : un unique écran de recherche (mon composant SearchBox) mène, après validation, à un écran à deux zones — une colonne de liste de personnages (CharacterList) sur la gauche, que j'ai dimensionnée à 4 colonnes Bulma sur 12, et une colonne de détail (CharacterDetails) sur la droite, sur les 8 colonnes restantes — complétée en dessous par une zone à onglets (BottomSection) que j'ai construite pour les aptitudes, les reliques et l'arbre de traces.

C'est une hiérarchie d'écrans que j'ai voulue simple et cohérente, et que je peux formaliser sans toucher au code existant.

Astral Database — schéma d'enchaînement des écrans (reconstitué)

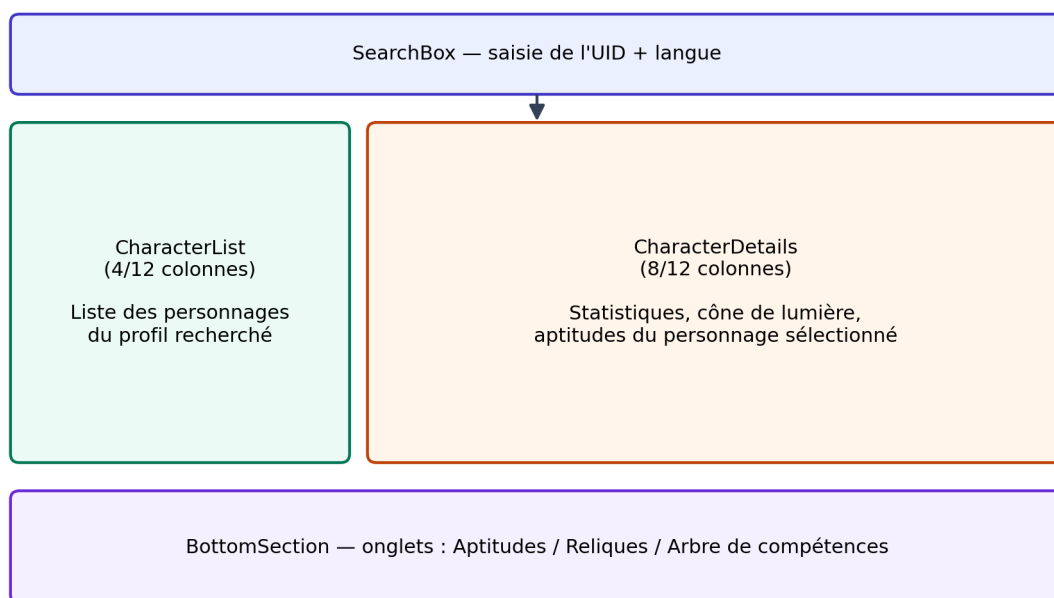
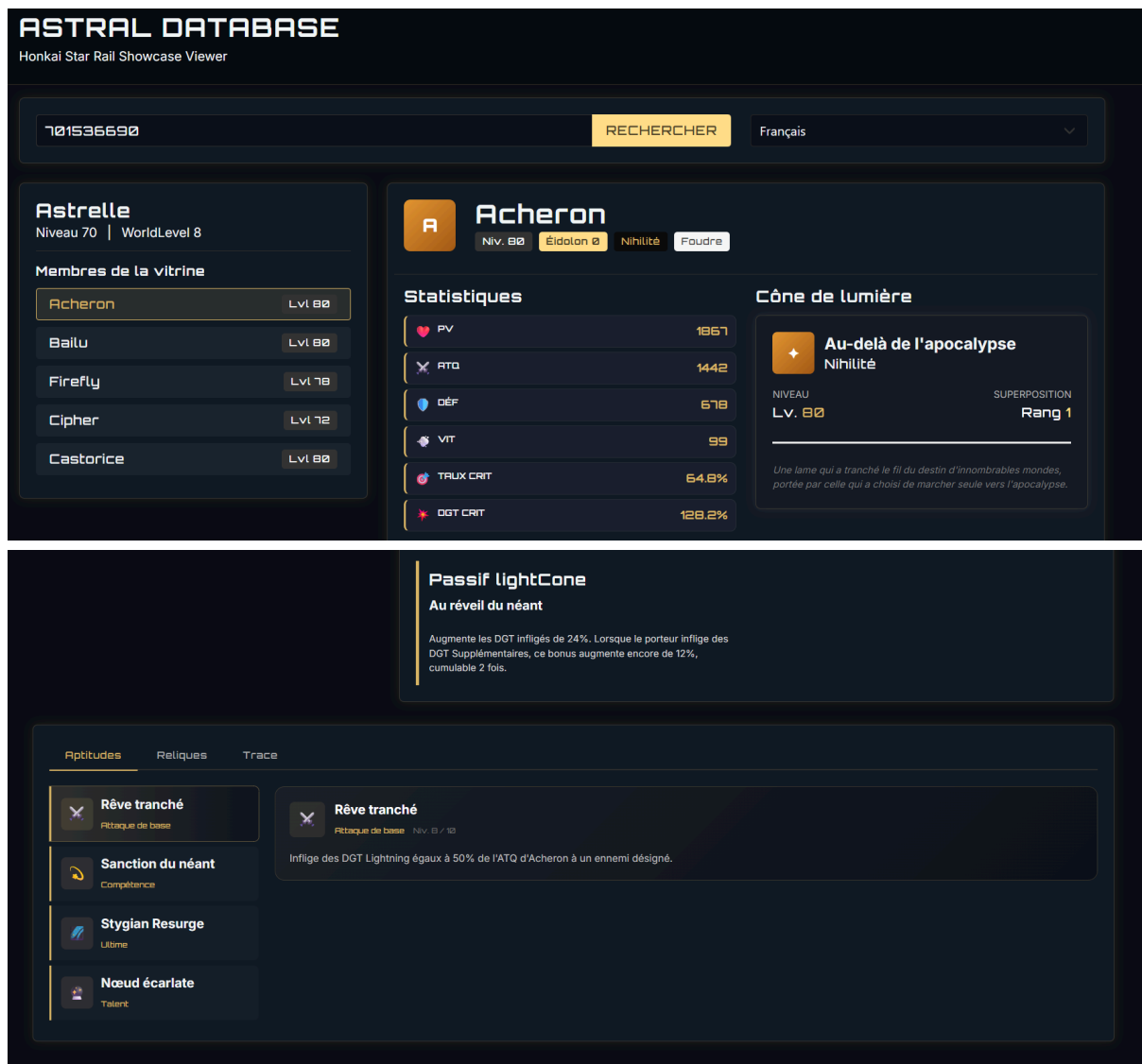


Figure — Schéma d'enchaînement des écrans reconstitué a posteriori (recherche → liste/détail → onglets).

Réaliser des interfaces utilisateur statiques web ou web mobile

J'ai construit l'ensemble de l'habillage visuel de mon application sur le framework CSS Bulma pour la structure (grille, boutons, tags, box), que j'ai complété par un fichier de surcharges App.css définissant l'identité graphique propre à mon projet via des variables CSS personnalisées que j'ai déclarées dans :root, par exemple --hsr-bg, --hsr-gold, --hsr-rarity-5 (un dégradé que j'ai créé selon la rareté d'un objet).

J'ai choisi cette approche par variables CSS pour centraliser ma palette de couleurs et la réutiliser de façon cohérente dans l'ensemble de mes composants, sans dupliquer les valeurs hexadécimales dans chaque règle — un choix qui m'a fait gagner beaucoup de temps quand j'ai voulu ajuster une teinte globalement.



J'ai complété Bulma par plusieurs classes utilitaires que j'ai écrites moi-même pour obtenir le rendu que je recherchais : has-border-bottom-hsr et has-border-left-hsr pour les séparateurs dorés

semi-transparents, equipment-icon-frame avec des dégradés que j'ai différenciés par rareté (rarity-5, rarity-4, rarity-3), ou encore bottom-tabs-nav / bottom-tab-btn pour le système d'onglets que j'ai construit en bas de la fiche personnage, avec un indicateur actif que j'anime par transition CSS sur la couleur et la bordure inférieure.

J'ai fait reposer la responsivité de mon affichage sur le système de colonnes Bulma, nativement adaptatif.

J'utilise columns is-mobile pour forcer le maintien de deux colonnes même sur petit écran dans le récapitulatif de niveau et de superposition d'un cône de lumière, et columns is-multiline pour que ma grille de reliques passe à la ligne selon la largeur disponible, avec is-half-tablet is-one-third-widescreen.

J'ai aussi traité le nettoyage du contenu textuel avant affichage côté statique : ma fonction sanitizeName, que j'ai dupliquée volontairement dans CharacterList.jsx et dans useCharacterDetails.jsx, retire les balises de mise en forme propres au moteur du jeu (<unbreak>) et les marqueurs de genre grammatical ({F#...}{M#...}) avant affichage, pour que les noms de personnages localisés en français s'affichent proprement quel que soit le genre du personnage — un détail auquel je n'avais pas pensé avant de tomber dessus en testant avec plusieurs personnages féminins et masculins.

Un point d'attention que j'ai identifié pour cette compétence concerne l'accessibilité : le référentiel demande explicitement la prise en compte du RGAA.

J'ai pensé à ajouter des attributs alt sur mes images (icônes de personnages, cônes de lumière, cartes de compétences), ce qui constitue une base correcte, mais je n'ai pas encore ajouté d'attributs aria-* (aria-label, aria-live), ni géré explicitement la navigation au clavier pour mes onglets, ni ajouté de lien d'évitement vers le contenu principal.

Je reviens sur ce point plus loin dans ce dossier.

Composants statiques secondaires : reliques et statistiques

J'ai décliné le même système de mise en page pour d'autres natures de contenu avec mes composants `RelicCard.jsx` et `RelicSection.jsx` : une relique s'affiche sous forme de box Bulma avec un en-tête (nom, niveau sous forme de tag +N), une statistique principale mise en avant et une liste de statistiques secondaires que j'affiche en colonne, séparée visuellement par une bordure pointillée que j'ai définie dans `App.css`.

Pour mon composant `CharacterStats.jsx`, utilisé dans la vue reliques, j'ai repris la même logique de carte que dans mon `StatsPanel` de `CharacterDetails.jsx`, mais avec un jeu de couleurs et d'icônes emoji dédié à chaque statistique.

Je me suis rendu compte en relisant mon code que j'ai dupliqué ce dictionnaire d'icônes entre deux fichiers (`useCharacterDetails.jsx` et `CharacterStats.jsx`) plutôt que de le mutualiser — une évolution que je garde en tête pour la suite.

J'ai ajouté un garde-fou défensif dans `LightConeCard.jsx` : si aucun cône de lumière n'est équipé par le personnage (ce qui peut arriver côté API), mon composant retourne immédiatement un paragraphe informatif plutôt que de tenter d'accéder aux propriétés d'un objet inexistant, ce qui m'a évité un plantage de l'application que j'ai rencontré une première fois en développement (l'erreur classique « `Cannot read properties of undefined` »).

J'applique ce type de vérification systématiquement en tête de ces composants d'affichage, sur `RelicCard`, `CharacterDetails` et `BottomSection` notamment.

Développer la partie dynamique des interfaces utilisateur web ou web mobile

J'ai construit toute la partie dynamique de mon application avec des hooks React personnalisés qui séparent ma logique métier de l'affichage : chaque composant visuel que j'écris est associé à un hook dédié qui centralise l'état, les calculs et les transformations de données, en laissant au composant la seule responsabilité du rendu JSX.

J'ai adopté cette architecture dès le début du projet parce qu'elle me permet de relire et de faire évoluer ma logique indépendamment du rendu visuel, ce qui s'est avéré précieux quand j'ai dû modifier plusieurs fois la façon dont j'affiche l'arbre de compétences sans toucher au calcul sous-jacent.

Gestion des appels réseau et de l'état applicatif global (App.jsx)

Dans mon composant racine App.jsx, je gère cinq états distincts avec useState : l'UID recherché, la langue sélectionnée, le profil complet renvoyé par l'API, l'index du personnage actuellement sélectionné et un indicateur de chargement.

J'ai mémorisé ma fonction loadProfile avec useCallback pour éviter qu'elle soit recréée à chaque rendu ; elle effectue l'appel fetch vers mon backend, gère le cas d'erreur réseau ou HTTP avec un message que j'affiche à l'utilisateur, puis réinitialise l'index sélectionné à 0 pour ne pas laisser affiché un personnage obsolète après le chargement d'un nouveau profil.

J'ai aussi ajouté un useEffect qui déclenche automatiquement une recherche au montage du composant si un UID est déjà présent dans le champ, pour éviter à l'utilisateur d'avoir à cliquer sur Rechercher au tout premier chargement de la page.

Le hook useSkillTree : la logique la plus complexe que j'ai écrite sur ce projet

Mon hook useSkillTree, utilisé par le composant SkillTreePanel, concentre la logique dynamique la plus riche de tout le projet.

Il reçoit en entrée l'arbre de compétences brut d'un personnage, son chemin (path) et la liste de ses compétences, et je lui fais retourner l'ensemble des données nécessaires au rendu du schéma SVG de l'arbre de traces : positions des nœuds, connexions entre nœuds, pourcentage de progression, nœud actuellement sélectionné et ses détails formatés.

J'ai écrit ma fonction getNodeStyle pour déterminer dynamiquement la couleur, la taille et la forme d'un nœud en analysant la chaîne de caractères de son icône (par exemple la présence de "_skill.", "ultimate" ou "_talent." dans le chemin du fichier).

J'ai dû procéder ainsi car l'API Mihomo ne fournit pas de type explicite pour chaque nœud de l'arbre de compétences.

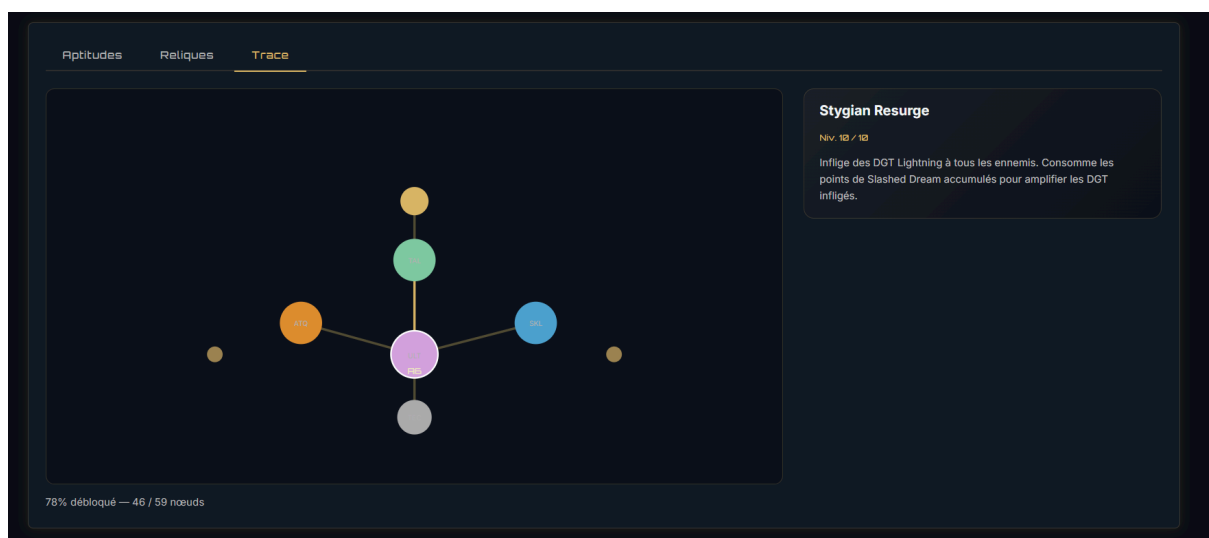
C'est une solution que je sais fragile, mais c'est la seule dont je disposais en l'absence de métadonnées structurées côté API — un choix que j'assume et que je peux justifier en entretien par les contraintes de la source de données externe que je ne contrôle pas.

J'ai écrit ma fonction `buildGroupedForms` pour gérer un cas que je n'avais pas anticipé au début du projet : certains personnages possèdent plusieurs formes pour une même compétence, par exemple une attaque de base normale et une attaque de base améliorée après activation d'une compétence spéciale.

Ma fonction filtre le tableau de compétences par type correspondant au nœud sélectionné et renvoie un indicateur `isGrouped` à `true` si plusieurs formes existent, ce qui déclenche dans mon composant `SkillCard.jsx` un rendu en accordéon qui liste chaque forme séparément plutôt qu'un affichage simple.

```
const buildGroupedForms = (node, allSkills) => {
  if (!node || !allSkills?.length) return null;
  const targetType = getSkillTypeForNode(node);
  const matching = allSkills.filter((s) => s.type === targetType);
  if (matching.length === 0) return null;
  return {
    isGrouped: matching.length > 1,
    forms: matching,
    primarySkill: matching[0],
  };
};
```

J'ai aussi codé le calcul du pourcentage de progression de l'arbre (ma variable `pct`) comme un exemple de manipulation dynamique des données : je compare chaque nœud à son niveau maximal (`maxLevel` ou `max_level` selon la version de la réponse API — je gère les deux formats de clé avec un opérateur OR logique pour assurer la compatibilité), puis j'arrondis et j'affiche en temps réel le ratio de nœuds entièrement débloqués sur le total.



Gestion des onglets et de la sélection dans BottomSection / useBottomSection

Dans mon hook useBottomSection, je classe dynamiquement les compétences d'un personnage en trois catégories (compétences principales, compétences de mémo-sprite, compétences spéciales) à partir d'ensembles JavaScript que j'ai définis pour chaque catégorie, puis je regroupe les compétences principales par type en détectant automatiquement si plusieurs formes existent pour un même type, de la même façon que dans useSkillTree.

Dans mon composant BottomSection.jsx, je gère un état local supplémentaire, selectedSkillId, que je réinitialise explicitement à null lors d'un changement d'onglet : sans cette précaution que j'ai ajoutée après avoir repéré le bug, le panneau de détail d'une compétence pouvait rester affiché alors qu'elle n'existait pas dans le nouvel onglet actif.

Formatage dynamique du texte enrichi (useCharacterDetails)

J'ai écrit ma fonction sanitizeAndFormatDescription, définie dans useCharacterDetails.jsx et que je réutilise dans plusieurs composants (BottomSection, SkillCard, CharacterDetails), pour transformer un texte brut contenant des balises propriétaires du jeu (`<color=#RRGGBB>texte</color>`, retours à la ligne encodés en `\n`) en une arborescence de nœuds React réels.

Je parcours le texte par expression régulière pour extraire les segments colorés et je construis des éléments `<span>` avec un style inline correspondant à la couleur hexadécimale extraite, en conservant l'ordre et les segments de texte neutre environnants.

C'est l'exemple que je choisirais pour illustrer mon développement dynamique côté client : je transforme une chaîne de caractères en interface utilisateur structurée, sans recharger la page ni faire d'appel serveur supplémentaire.

L'ensemble de ces traitements dynamiques s'exécute exclusivement côté client.

Je n'ai pas eu besoin d'échapper manuellement ce contenu car React le fait pour moi par défaut : JSX échappe automatiquement le contenu inséré via les expressions `{...}`, et je n'utilise nulle part `dangerouslySetInnerHTML` dans mon projet.

En construisant mes éléments colorés avec des `<span>` React plutôt qu'avec `innerHTML`, je limite structurellement le risque d'injection de script (XSS) par rapport à un rendu HTML brut, même si je considère la source de mes données (l'API Mihomo) comme fiable a priori.

Résolution dynamique des positions de l'arbre (pathLayouts) et des assets (imageMap)

L'arbre de compétences d'un personnage est un graphe dont l'API Mihomo ne fournit ni coordonnées d'affichage, ni tracé des connexions entre nœuds : seules l'existence des nœuds et leur niveau me sont renvoyés.

J'ai résolu ce problème avec mon fichier `pathLayouts.js`, en associant à chacune des neuf voies du jeu un fichier de configuration statique dédié (`Destruction.js`, `Harmony.js`, `Nihility.js`...) où je fixe manuellement les coordonnées x/y de chaque point d'ancrage ainsi que la liste des connexions à tracer entre eux.

Ma fonction `getLayout`, dans `useSkillTree.js`, recherche ensuite la disposition correspondant à la voie du personnage sélectionné, avec une résolution en cascade que j'ai rendue tolérante à la casse et aux variantes de nommage, avant de replier sur la disposition « Destruction » par défaut — ce qui absorbe les incohérences de nommage que j'ai observées entre les différentes versions de l'API au fil de mes tests.

De façon symétrique, mon fichier `imageMap.js` résout à la volée les chemins d'accès à mes images locales (icônes de personnages, de compétences, de traces) à partir de l'identifiant numérique d'un personnage.

Je fusionne par décomposition d'objets les dictionnaires d'assets propres à chaque personnage que j'ai implémenté en un catalogue unique, interrogé par des fonctions d'accès que j'ai sécurisées (`getCharacterImages`, `getSkillIcon`) : elles renvoient systématiquement `null` plutôt qu'une exception si l'identifiant demandé n'existe pas encore dans mon catalogue local.

Ce mécanisme me permet de continuer à faire fonctionner mon frontend correctement, avec un repli sur une icône générique, même pour un personnage dont je n'ai pas encore ajouté les assets au projet.

Un dernier détail auquel j'ai fait attention : SearchBox.jsx

Mon composant SearchBox.jsx est volontairement le plus simple de mon projet, mais j'y ai quand même fait des choix que je peux justifier.

Je l'ai conçu comme un composant purement contrôlé : il ne détient aucun état local, il reçoit ses valeurs (userId, language) et ses fonctions de mise à jour directement en props depuis App.jsx.

J'ai fait ce choix pour que mon composant racine reste la seule source de vérité sur l'état de recherche, ce qui m'évite d'avoir à synchroniser un état local avec l'état global — un piège classique que j'ai appris à éviter en cours de formation.

J'ai aussi fait en sorte que le bouton Rechercher affiche un indicateur de chargement Bulma (is-loading) directement dérivé de mon état loading défini dans App.jsx, sans logique supplémentaire dans le composant lui-même.

Jeu d'essai que j'ai réalisé — Astral Database

Conformément à ce que demande le référentiel, j'ai réalisé un jeu d'essai fonctionnel sur ma fonctionnalité de recherche de profil (SearchBox → App.jsx → loadProfile).

Voici les cas que j'ai testés :

Donnée en entrée	Résultat attendu	Résultat obtenu
UID valide existant (ex. 701536690), langue fr	Profil chargé, liste de personnages affichée, premier personnage sélectionné	Conforme
UID inexistant ou invalide	Message d'erreur affiché (« Profil introuvable ou erreur API »), aucune donnée obsolète conservée	Conforme
Champ UID vide, clic sur Rechercher	Message d'erreur local (« Veuillez entrer un UID valide »), aucun appel réseau déclenché	Conforme
Changement de langue après un premier chargement réussi	Rechargement du profil dans la nouvelle langue via le useEffect dépendant de loadProfile	Conforme

Complément : la couche serveur que j'ai construite pour Astral Database

Même si j'utilise principalement Astral Database pour illustrer l'activité front-end, mon backend Express contient des composants métier côté serveur assez proches de ceux que je détaille plus longuement dans la partie II à partir de mon projet Cahier de recettes collaboratif.

Je les présente ici brièvement pour mémoire ; je pourrai les évoquer en entretien technique comme second exemple de composant serveur si le jury le souhaite.

J'ai écrit mon module `utils/serializers/character.js` pour transformer la réponse brute et peu structurée de l'API Mihomo en un objet propre et prévisible pour mon frontend : je convertis systématiquement les identifiants en chaînes de caractères, je définis des valeurs par défaut explicites (null plutôt que undefined), et j'ai écrit une fonction `getNodeType` qui classe chaque nœud d'arbre de compétences selon des motifs présents dans le nom de fichier de son icône.

En la relisant, je me suis rendu compte que j'ai dupliqué cette logique côté frontend et côté backend, ce qui constitue une répétition de règles métier que je pourrais mutualiser dans une bibliothèque partagée si je poursuis ce projet.

J'ai mis en place, dans mon module `config/lightConeCache.js`, un cache mémoire avec expiration (durée de vie de 24 heures, que je calcule par comparaison de timestamps) pour les descriptions des cônes de lumière, que je charge depuis un dépôt statique GitHub tiers.

J'ai ajouté ce cache pour éviter de solliciter ce dépôt à chaque requête d'un utilisateur de mon application, une préoccupation de performance et de sobriété réseau à laquelle je n'avais pas pensé avant de voir les temps de réponse augmenter en test.

J'ai aussi pris soin de gérer les erreurs de chargement sans interrompre mon service : si le cache échoue à se charger, mon application continue de fonctionner avec des descriptions vides plutôt que de renvoyer une erreur 500 au frontend.

Ce que je retiens de ce projet

Astral Database est le projet où j'ai le plus travaillé la couche de présentation et l'expérience utilisateur, à travers un thème visuel complet, une navigation par onglets et un rendu dynamique de données complexes comme l'arbre de compétences.

C'est aussi le projet qui m'a le plus confronté à l'incertitude d'une source de données que je ne maîtrise pas : j'ai dû accepter de construire mes composants de façon défensive, avec des vérifications systématiques et des valeurs de repli, plutôt que de supposer que l'API me renverrait toujours exactement ce que j'attendais.

Si je devais reprendre ce projet aujourd'hui, je commencerais probablement par écrire mes maquettes avant le code, ce qui m'aurait sans doute évité quelques allers-retours dans l'organisation de mes colonnes Bulma.

Sécurité et bonnes pratiques — Astral Database

Côté React, mon risque XSS est structurellement limité par le fonctionnement de JSX, qui échappe automatiquement le contenu que j'insère via les expressions `{...}`, et je n'utilise nulle part `dangerouslySetInnerHTML`.

Comme indiqué en CP1, j'ai codé en dur l'URL de mon API backend dans `App.jsx` (`http://localhost:5000/api/user/...`) plutôt que de la lire depuis une variable d'environnement ; c'est un point que je dois corriger avant toute mise en production.

Chapitre 2 — Cahier de recettes collaboratif (contribution individuelle, back-end)

Pour cette partie, je m'appuie sur mon Cahier de recettes collaboratif (Node.js, Express, MongoDB), qui me permet de démontrer l'essentiel des compétences d'accès aux données NoSQL, de développement de composants métier côté serveur et de documentation du déploiement.

Présentation du projet : Cahier de recettes collaboratif

J'ai conçu le Cahier de recettes collaboratif comme une API REST permettant à des utilisateurs inscrits de publier des recettes de cuisine (titre, liste d'ingrédients, instructions), de les consulter, de les modifier ou de les supprimer s'ils en sont l'auteur, de les rechercher par ingrédient, et d'y ajouter des commentaires.

J'ai voulu, avec ce projet, aller plus loin que les exercices guidés de ma formation en construisant une application complète avec une vraie logique d'autorisation (pas seulement d'authentification) et une documentation d'API que je pourrais montrer à un recruteur.

J'ai construit mon application sur Node.js et le framework Express 5, avec une base de données MongoDB Atlas hébergée dans le cloud, que je pilote avec l'ODM Mongoose.

J'assure l'authentification par jetons JWT combinés au hachage des mots de passe avec bcrypt.

J'ai aussi pris le temps de générer automatiquement la documentation de mon API à partir d'annotations JSDoc, grâce à swagger-jsdoc, et de l'exposer via une interface interactive Swagger UI — un choix qui me permet de tester mes propres routes sans écrire de client dédié, et qui donne au dossier une preuve tangible de ma capacité à documenter mon travail.

```
// Swagger
const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'API Cahier de Recettes Collaboratif',
      version: '1.0.0',
      description: 'API pour gérer les recettes de cuisine',
    },
    servers: [
      {
        url: 'http://localhost:3000',
      },
    ],
    components: {
      securitySchemes: {
        bearerAuth: {
          type: 'http',
          scheme: 'bearer',
          bearerFormat: 'JWT',
        },
      },
    },
  },
  apis: ['./routes/*.js'], // files containing annotations
};

const swaggerSpec = swaggerJsdoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

Voici comment j'ai organisé les fichiers de ce projet :

```
server.js           - mon point d'entrée, montage de Swagger, montage des routes
config/db.js        - ma connexion Mongoose à MongoDB Atlas
models/Recipe.js     - mon schéma Mongoose recette (+ commentaires imbriqués)
models/User.js       - mon schéma Mongoose utilisateur
controllers/recipeController.js - ma logique métier recettes (CRUD, recherche, commentaires)
controllers/userController.js - inscription, connexion, émission JWT
middlewares/auth.js  - vérification du jeton JWT sur mes routes protégées
routes/recipeRoutes.js - déclaration de mes routes /api/recipes
routes/userRoutes.js - déclaration de mes routes /api/users (+ annotations Swagger)
```

J'ai choisi cette organisation en couches (routes / contrôleurs / modèles) parce que c'est une structure que j'avais découverte en formation et que je voulais mettre en pratique sur un projet où j'ai une totale liberté de conception.

Ce que j'en retiens, c'est que cette séparation m'a vraiment aidé au moment d'ajouter la fonctionnalité de commentaires en cours de projet : je n'ai eu à modifier que mon contrôleur `recipeController.js` et mon schéma `Recipe.js`, sans toucher à mes routes ni à mon middleware d'authentification, que je réutilisais tel quel.

Je pense que c'est le signe que mon architecture initiale était suffisamment découplée.

Cahier de recettes collaboratif — architecture en couches

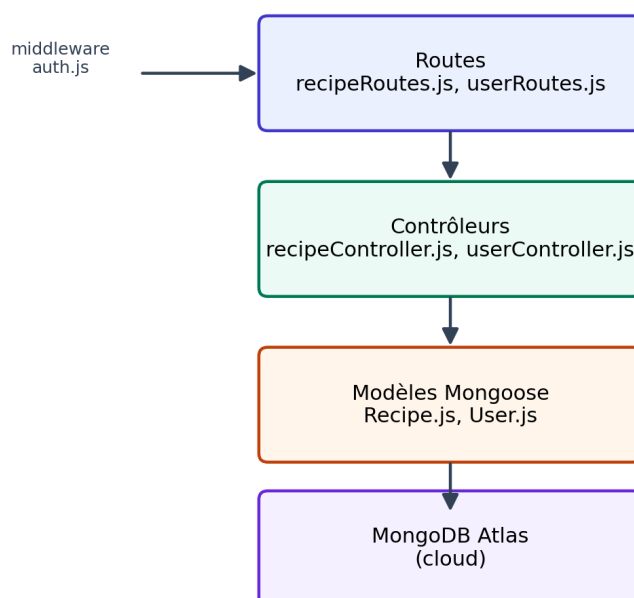


Figure — Architecture en couches du Cahier de recettes collaboratif.

Installer et configurer son environnement de travail (complément back-end)

J'ai encapsulé la connexion à MongoDB Atlas dans mon fichier `config/db.js`, dans une fonction `connectToMongoDB`, avec un indicateur booléen `isConnected` en fermeture du module pour éviter d'ouvrir plusieurs connexions simultanées si la fonction est appelée plusieurs fois au cours de la vie du processus — un bug que j'avais effectivement rencontré avant d'ajouter cette protection.

Je lis ma chaîne de connexion depuis une variable d'environnement (`MONGO_URI` ou, à défaut, `MONGODB_URI`), que je charge avec `dotenv` à partir d'un fichier `.env` que je ne versionne pas.

Si aucune de ces deux variables n'est présente, je lève une erreur explicite immédiatement au démarrage, plutôt que de laisser mon application échouer plus tard de façon moins compréhensible.

```
await mongoose.connect(mongoURI, {
  serverApi: { version: "1", strict: true, deprecationErrors: true },
  tls: true,
  retryWrites: true,
  w: "majority",
  serverSelectionTimeoutMS: 30000,
  family: 4,
});
```

J'ai choisi mes options de connexion avec soin, plutôt que de garder les valeurs par défaut : `tls: true` impose une connexion chiffrée à mon cluster MongoDB Atlas ; `retryWrites: true` et `w: "majority"` garantissent qu'une écriture n'est considérée confirmée qu'une fois répliquée sur la majorité des nœuds du cluster, ce qui réduit le risque de perte de données en cas de bascule ; `family: 4` force la résolution DNS en IPv4, un correctif que j'ai dû ajouter après avoir rencontré des échecs de résolution IPv6 vers les clusters Atlas depuis mon environnement.

J'ai défini deux scripts dans mon `package.json` : `start` (`node server.js`, pour la production) et `dev` (`nodemon server.js`, pour le développement avec rechargement automatique).

Mes dépendances de production couvrent l'ensemble des besoins fonctionnels de mon projet : `express`, `cors`, `mongoose`, `bcryptjs`, `jsonwebtoken`, `dotenv`, `express-validator`, ainsi que `swagger-jsdoc` et `swagger-ui-express` pour ma documentation.

J'ai versionné mon projet avec Git, en excluant mon fichier `.env` du suivi de version pour ne pas exposer ma chaîne de connexion à la base de données ni mon secret de signature JWT.

CP6 (volet NoSQL) — Développer des composants d'accès aux données NoSQL

Je démontre cette compétence sur son volet NoSQL avec mon Cahier de recettes collaboratif.

Volet NoSQL — Cahier de recettes collaboratif

J'ai défini mes schémas Mongoose pour structurer les documents stockés en base.

Mon schéma Recipe comporte un titre et des instructions obligatoires, un tableau d'ingrédients, une référence vers l'auteur (que j'ai établie avec ObjectId et ref: 'User', créant une relation logique entre mes deux collections bien que MongoDB ne l'impose pas au niveau moteur comme le ferait une clé étrangère relationnelle) et un tableau de commentaires que j'ai choisi d'imbriquer directement dans le document. J'ai adopté ce choix de modélisation typique du NoSQL orienté documents — privilégier l'imbrication des données consultées ensemble plutôt que la relation entre plusieurs collections — parce qu'il me permet d'afficher une recette avec ses commentaires en une seule requête, plutôt que d'en faire deux.

Pour mon accès en lecture avec jointure logique, j'utilise la méthode populate de Mongoose : `Recipe.find().populate('author', 'name')` remplace la référence ObjectId de l'auteur par le document utilisateur correspondant, en ne récupérant que son champ name. J'ai fait ce choix de projection explicite pour éviter de transmettre à mon frontend des informations sensibles comme le mot de passe haché ou l'email de l'auteur d'une recette que l'utilisateur n'a pas consultée volontairement.

Pour ma fonctionnalité de recherche (`searchRecipes`), je construis dynamiquement un objet de requête Mongoose selon les paramètres reçus en query string : si un paramètre ingredient est présent, j'ajoute une condition de correspondance par expression régulière insensible à la casse sur le champ ingredients ; si un paramètre sort=recent est présent, j'applique un tri sur le champ createdAt en ordre décroissant. J'ai résolu ce problème de filtrage optionnel et cumulatif avec l'idiome propre à MongoDB, de la même façon que j'ai résolu le problème équivalent côté SQL dans `recherche.php` avec mon `WHERE 1=1` dynamique — je trouve intéressant d'avoir eu à traiter le même besoin fonctionnel avec deux technologies d'accès aux données différentes.

Développer des composants métier côté serveur

J'ai organisé la logique métier de mon Cahier de recettes collaboratif selon une architecture en couches classique : mes routes (`recipeRoutes.js`, `userRoutes.js`) déclarent les points d'entrée HTTP et les middlewares applicables, mes contrôleurs (`recipeController.js`, `userController.js`) contiennent la logique métier proprement dite, et mes modèles Mongoose (`Recipe.js`, `User.js`) définissent la structure et les contraintes de mes données. J'exporte chaque fonction de contrôleur individuellement et je l'encapsule dans un bloc `try/catch`, avec un code de statut HTTP explicite que j'adapte à la nature de l'erreur : 400 pour une erreur de validation, 403 pour un refus d'autorisation, 404 pour une ressource introuvable, 500 pour une erreur serveur inattendue.

Authentification et sécurisation des mots de passe

Dans mon processus d'inscription (`userController.register`), je vérifie d'abord l'absence de compte existant pour l'adresse email fournie, avant de créer le nouvel utilisateur.

Je ne stocke jamais le mot de passe en clair : je génère un sel avec `bcrypt.genSalt(10)` — dix tours de hachage, un compromis que j'ai retenu entre robustesse cryptographique et temps de calcul acceptable pour l'utilisateur — puis je hache le mot de passe avec ce sel avant de le persister. Pour ma connexion (`userController.login`), je compare le mot de passe fourni au hachage stocké via `bcrypt.compare`, qui recalcule le hachage avec le même sel et compare les résultats, sans jamais avoir besoin de déchiffrer quoi que ce soit.

Dans les deux cas, inscription ou connexion réussie, je signe un jeton JWT avec `jsonwebtoken.sign`, en y plaçant l'identifiant et le nom de l'utilisateur comme charge utile, une durée de validité que j'ai limitée à une heure, et un secret de signature que je lis depuis la variable d'environnement `JWT_SECRET` — je ne le code jamais en dur dans mon code source.

Cahier de recettes collaboratif — flux d'authentification



Figure — Flux d'authentification (hachage bcrypt, signature JWT, vérification par middleware).

Middleware d'authentification et contrôle d'accès

J'ai écrit mon middleware `middlewares/auth.js` comme un composant serveur transversal que je réutilise sur l'ensemble de mes routes nécessitant une authentification.

Il lit l'en-tête HTTP `Authorization`, renvoie une erreur 401 si celui-ci est absent, vérifie et décode le jeton avec `jwt.verify`, renvoie une erreur 400 si le jeton est invalide ou expiré, puis attache les informations décodées à `req.user` avant d'appeler `next()` pour poursuivre le traitement de la requête.

Je compose ensuite ce middleware directement dans la déclaration de mes routes, ce qui rend explicite, à la seule lecture du fichier, quelles actions nécessitent une authentification chez moi.

Au-delà de l'authentification, j'applique dans mes contrôleurs un contrôle d'autorisation propre à chaque ressource : mes fonctions `updateRecipe` et `deleteRecipe` vérifient que `recipe.author.toString()` correspond bien à `req.user.id` avant d'autoriser la modification ou la suppression, et renvoient un statut 403 dans le cas contraire.

J'ai ajouté cette vérification car mon middleware `auth` garantit seulement qu'un utilisateur est authentifié, pas qu'il est le propriétaire légitime de la ressource qu'il tente de modifier — une distinction entre authentification et autorisation à laquelle j'ai fait attention dès la conception, et que le référentiel attend explicitement.

```

exports.updateRecipe = async (req, res) => {
  const recipe = await Recipe.findById(req.params.id);

```



```

if (!recipe) return res.status(404).json({ message: "Recette non trouvée" });
if (recipe.author.toString() !== req.user.id)
  return res.status(403).json({ message: "Accès refusé" });
const updatedRecipe = await Recipe.findByIdAndUpdate(req.params.id, req.body, { new:
true });
res.json(updatedRecipe);
};

```

Documentation de l'API et communication technique

J'ai documenté mes routes utilisateur directement dans le code source, à l'aide de blocs de commentaires JSDoc suivant la spécification OpenAPI, que swagger-jsdoc interprète au démarrage de mon serveur pour générer une spécification consultable et testable via Swagger UI. J'ai choisi cette documentation vivante — mise à jour en même temps que mon code puisqu'elle réside dans les mêmes fichiers — pour faciliter la communication avec d'autres développeurs ou avec un client sur le fonctionnement exact de mon API, sans avoir à maintenir un document séparé qui risquerait de devenir obsolète.

Jeu d'essai que j'ai réalisé — Cahier de recettes collaboratif

J'ai construit un jeu d'essai portant sur la modification d'une recette (route PUT /api/recipes/:id), qui met en jeu à la fois mon authentification (middleware auth) et mon contrôle d'autorisation propre à la ressource.

Donnée en entrée	Résultat attendu	Résultat obtenu
Requête sans en-tête Authorization	Statut 401, message « Accès refusé »	Conforme
Jeton JWT expiré ou falsifié	Statut 400, message « Token non valide »	Conforme
Jeton valide, utilisateur non auteur de la recette	Statut 403, message « Accès refusé »	Conforme
Jeton valide, utilisateur auteur de la recette, corps de requête valide	Statut 200, recette mise à jour renvoyée	Conforme
Identifiant de recette inexistant	Statut 404, message « Recette non trouvée »	Conforme

Un choix de modélisation que je peux justifier : User.js et Recipe.js

J'ai volontairement gardé mon schéma `User.js` minimal (nom, email unique, mot de passe haché, date de création) plutôt que d'y ajouter dès le départ des champs que je n'utilisais pas encore, comme une biographie ou une photo de profil.

J'ai préféré démarrer simple et étendre mon schéma au fur et à mesure de mes besoins réels, plutôt que d'anticiper des fonctionnalités hypothétiques.

À l'inverse, j'ai enrichi mon schéma `Recipe.js` dès le début avec un tableau de commentaires imbriqués, car je savais dès la conception que cette fonctionnalité ferait partie de mon projet et qu'elle bénéficierait de la modélisation orientée documents de MongoDB.

Ce contraste entre mes deux schémas illustre, je crois, une réflexion de conception plutôt qu'une simple retranscription d'un cahier des charges : j'ai adapté ma structure de données à l'usage réel que j'en fais.

Documenter le déploiement d'une application dynamique web ou web mobile

J'ai bien prévu, dans le `package.json` de mon Cahier de recettes collaboratif, un script `start` (`node server.js`) qui constitue un point d'entrée minimal de déploiement, et je lis ma configuration (port, chaîne de connexion, secret JWT) depuis des variables d'environnement plutôt que de coder des valeurs en dur — une bonne pratique nécessaire, mais je sais qu'elle n'est pas suffisante au regard de ce qu'attend le référentiel.

Le référentiel attend explicitement que je rédige une procédure de déploiement et que j'écrive des scripts de déploiement documentés, en tenant compte des dépendances et des versions de mon application.

Je n'ai encore produit aucun de ces deux livrables : je n'ai pas de fichier `Dockerfile` ou `docker-compose.yml`, pas de configuration de pipeline d'intégration ou de déploiement continu, et pas de document décrivant mes étapes de mise en production.

C'est le point que je considère comme le plus urgent à traiter avant ma session d'examen, puisqu'il correspond à une compétence entière du référentiel sans preuve actuelle dans mon dossier ; je prévois de déployer réellement mon application (par exemple sur Render ou Railway) et de rédiger la procédure correspondante avant l'entretien.

Ce que je retiens de ce projet

En travaillant sur mon Cahier de recettes collaboratif, en parallèle de mon expérience en équipe sur MarsAI durant mon stage, j'ai pu mesurer la différence entre développer seul une application de bout en bout — où je maîtrise l'ensemble des choix, de la modélisation des données à l'implémentation — et contribuer à un projet de groupe où je ne maîtrise qu'une partie du périmètre technique, ici le frontend.

Ces deux expériences se complètent : l'une me permet de démontrer une maîtrise technique complète et autonome, l'autre un travail d'équipe structuré en méthode Agile.

Sécurité et bonnes pratiques — Cahier de recettes collaboratif

Authentification, hachage et gestion des sessions

Sur mon Cahier de recettes collaboratif, j'applique les pratiques attendues : mots de passe hachés avec bcrypt, jamais stockés ni transmis en clair, jetons JWT à durée de vie limitée signés avec un secret externalisé, et middleware de vérification systématique sur mes routes sensibles.

Je vois une amélioration possible, que je peux mentionner en entretien comme piste de progression plutôt que comme correction obligatoire : je pourrais transmettre mon jeton via un cookie httpOnly plutôt que dans l'en-tête Authorization géré manuellement côté client, ce qui réduirait l'exposition du jeton aux scripts côté navigateur en cas de faille XSS ailleurs dans l'application.

J'externalise correctement mes secrets dans un fichier .env non versionné (chaîne de connexion MongoDB, secret de signature JWT), conformément aux recommandations de l'ANSSI.

Validation des entrées utilisateur

J'ai ajouté le paquet express-validator aux dépendances de mon Cahier de recettes collaboratif, mais je ne l'utilise encore dans aucun de mes contrôleurs : ma validation des champs repose actuellement sur les seules contraintes que j'ai déclarées au niveau de mon schéma Mongoose, qui s'exécutent au moment de l'enregistrement en base mais ne produisent pas nécessairement des messages d'erreur exploitables côté client avant cet enregistrement.

J'envisage d'ajouter des middlewares express-validator explicites sur mes routes de création pour me mettre directement en conformité avec le critère du référentiel sur le contrôle et la validation des entrées.

Chapitre 3 — MarsAI (projet de groupe, contribution frontend)

Avant de démarrer Astral Database, j'ai participé durant mon stage de fin de formation au développement de MarsAI, un site web réalisé en groupe, sur lequel mon rôle était exclusivement centré sur le frontend (React).

Nous avons travaillé en méthode Agile pendant toute la durée du stage : un daily stand-up chaque matin pour synchroniser l'équipe sur l'avancement et les blocages, et une présentation de nos avancées au client à la fin de chaque semaine, ce qui nous obligeait à livrer des incréments fonctionnels régulièrement plutôt que de développer en silo pendant plusieurs semaines.

MarsAI — cycle de travail en méthode Agile

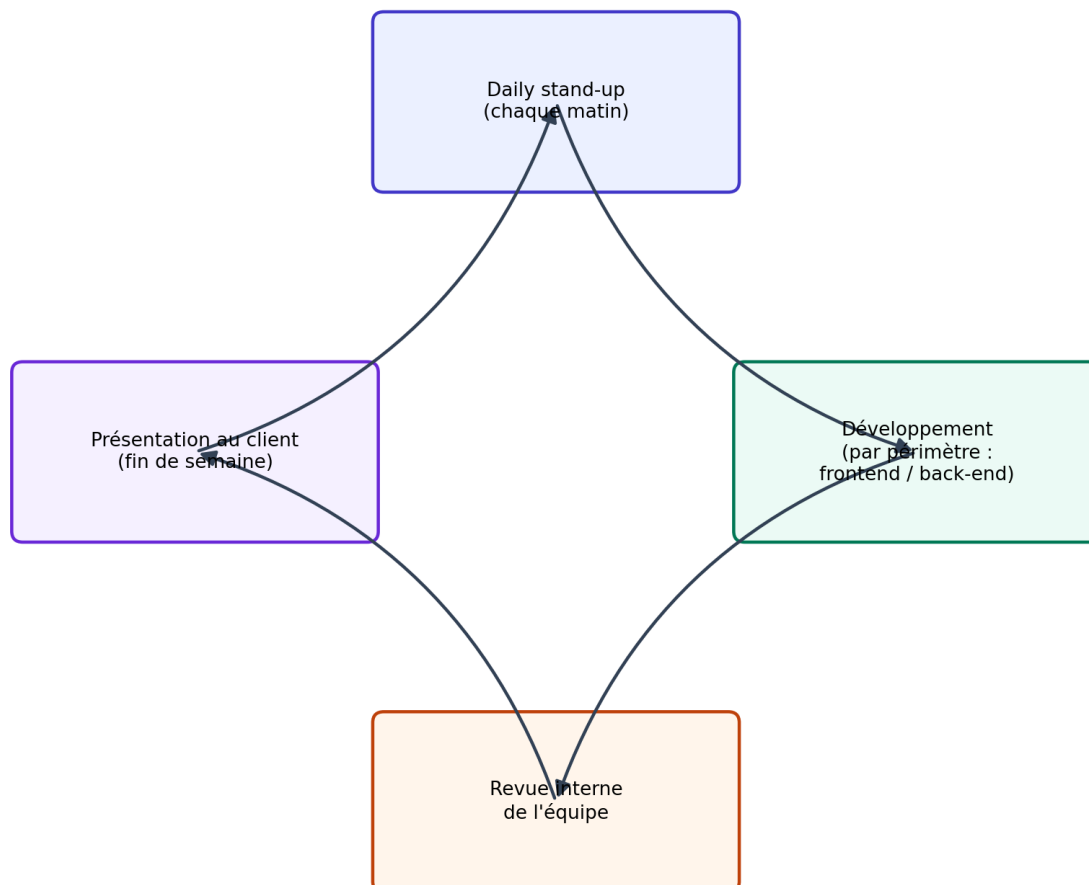
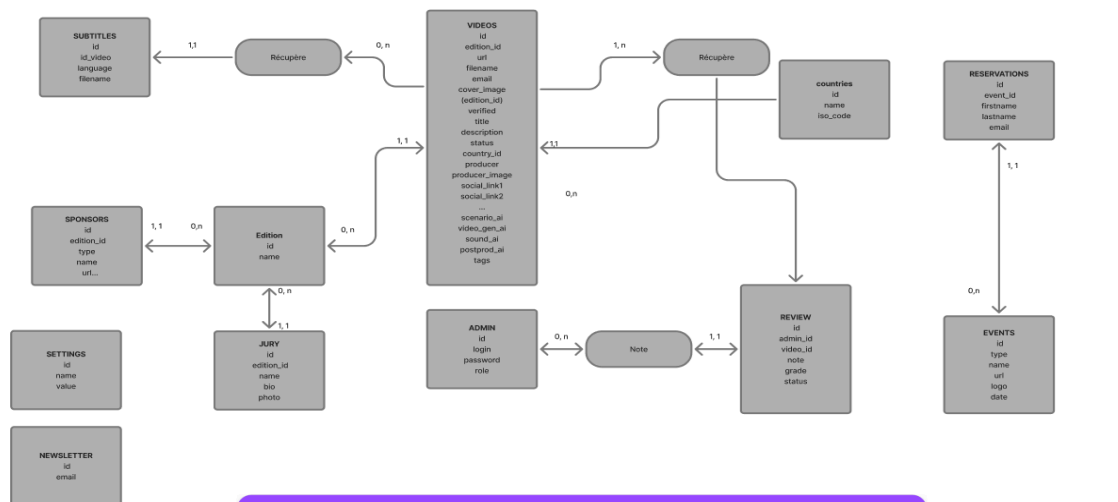


Figure — Cycle de travail en méthode Agile suivi durant le stage sur MarsAI.

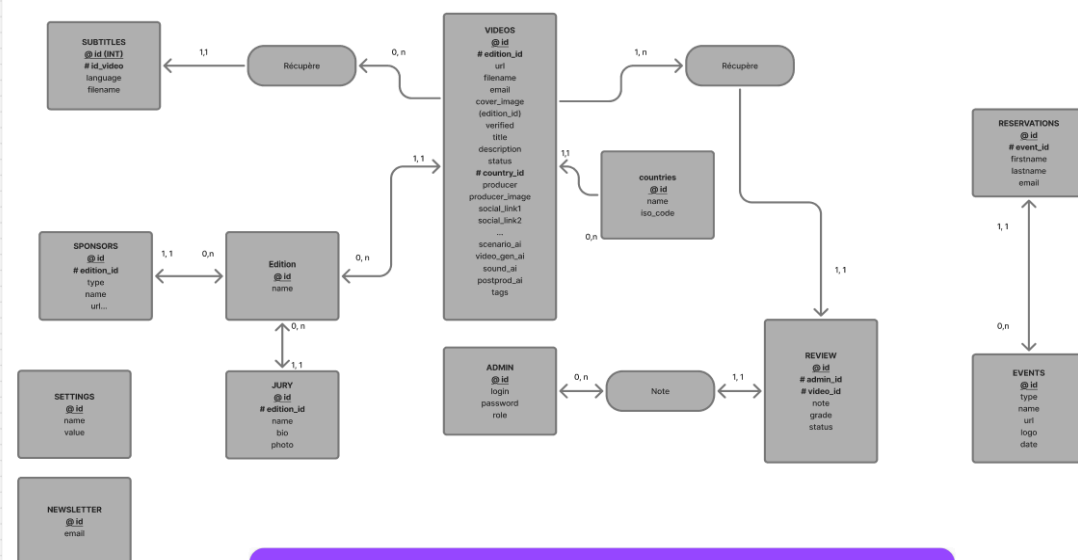
Conception collaborative : MCD, MLD et maquettes

En amont du développement, nous avons conçu ensemble le modèle conceptuel de données (MCD) et le modèle logique de données (MLD) du projet, en identifiant les entités (événements, réservations, utilisateurs, contenus de galerie, messages de contact, abonnés à la newsletter) et leurs relations. J'ai participé à ces ateliers de conception avec le reste de l'équipe ; ce travail est disponible [ici](#) :

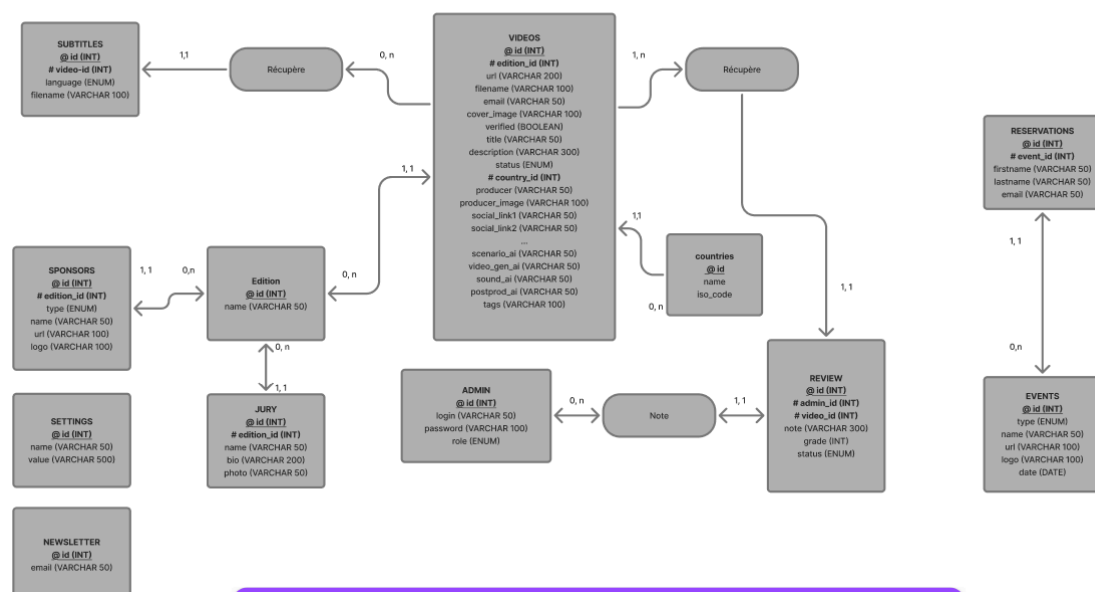
MCD



MLD



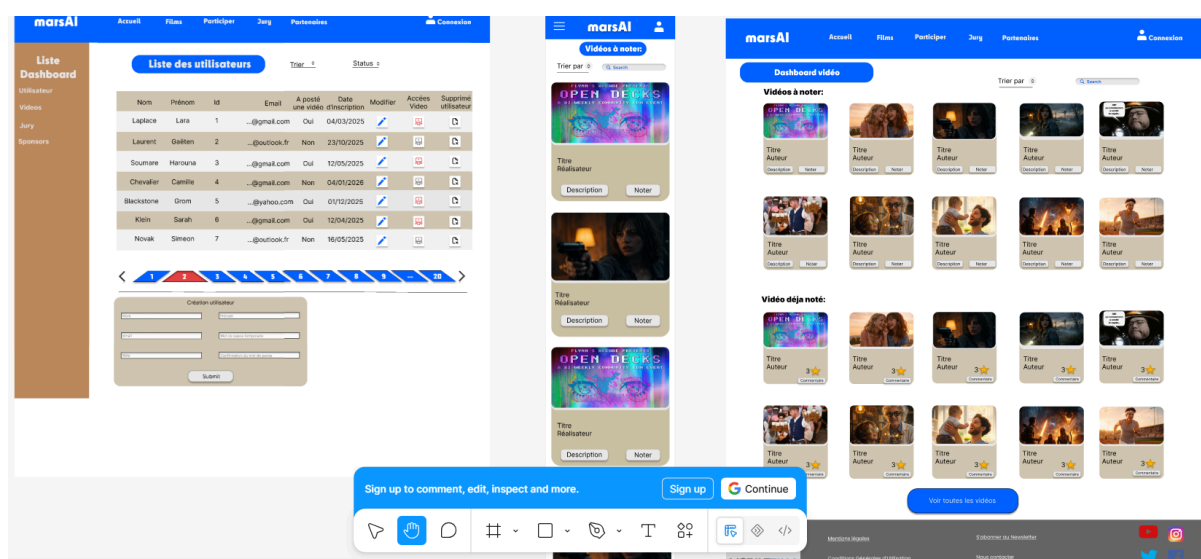
MPD



<https://www.figma.com/board/2draydFu2xRPptZczqRbwv/Use-Cases---marsAI>.

Je précise que je n'ai pas développé moi-même la couche de persistance ni les requêtes SQL correspondantes — cette partie a été prise en charge par d'autres membres de l'équipe en charge du back-end — mais j'ai contribué à la réflexion sur la structure des données, en particulier pour m'assurer que les objets que je recevais côté frontend (événements, réservations, messages) correspondaient à ce qui était réellement modélisé en base.

Nous avons également maqueté l'ensemble des écrans du site sur Figma avant de commencer à coder, une maquette à laquelle j'ai participé :



<https://www.figma.com/design/YF2mZu4yAEmlxF32XZycN0/maquette-marsAi?node-id=0-1&p=f>.

Cette étape de maquetage collective, que je n'ai pas menée seul mais dont j'ai fait partie, m'a permis de développer mes composants React directement à partir d'une base visuelle validée par l'équipe et par le client, à l'inverse d'Astral Database où j'ai itéré directement dans le code faute de maquette préalable.

Mes composants frontend

Sur la partie frontend, j'ai développé plusieurs composants et pages du site.

Mon composant Header.jsx gère la navigation principale : un menu responsive qui bascule entre une barre de navigation horizontale sur desktop et un menu plein écran animé sur mobile, un changement de style au scroll (fond opaque avec flou d'arrière-plan après 20 pixels de défilement), un sélecteur de langue français/anglais synchronisé avec i18next, et une liste de navigation qui s'adapte dynamiquement selon une variable de phase du projet (settings.phase) fournie par un contexte React global.

Mon composant Footer.jsx repose sur cette même logique pilotée par le contexte des réglages (useSettings) et propose un formulaire d'inscription à la newsletter que j'ai validé côté client par expression régulière avant l'envoi à l'API (/api/newsletter/subscribe), ainsi que les liens vers les réseaux sociaux et la navigation du site, avec un contenu éditable dynamiquement via un composant Editable mis en place par l'équipe pour permettre au client de modifier certains textes sans toucher au code.

J'ai également implémenté le bandeau de consentement aux cookies (Cookie.jsx), conforme aux exigences RGPD : à la première visite, l'utilisateur peut accepter ou refuser les cookies, son choix étant persisté dans le localStorage sous une clé dédiée (marsai_cookie_consent_status) pour ne plus lui présenter la bannière tant qu'il n'a pas réinitialisé son navigateur.

J'ai ajouté une étape de confirmation explicite en cas de refus, pour m'assurer que l'utilisateur comprend bien les conséquences de son choix avant de le valider.

Sur la page des événements (Event_page.jsx), j'ai construit un système de liste/détail : la liste des événements est récupérée par un appel fetch à l'API (/api/events) au montage du composant, chaque événement étant affiché par mon composant EventButton.jsx et sélectionnable pour afficher son détail dans SelectedEvent.jsx (date, lieu, durée, description, lien externe).

J'ai ajouté un composant RegistrationForm.jsx qui permet à un visiteur de s'inscrire à l'événement sélectionné, avec validation des champs obligatoires côté client avant l'envoi de la requête POST vers /api/reservations, et un affichage temporisé (trois secondes) des messages de succès ou d'erreur pour ne pas surcharger l'interface une fois l'action terminée.

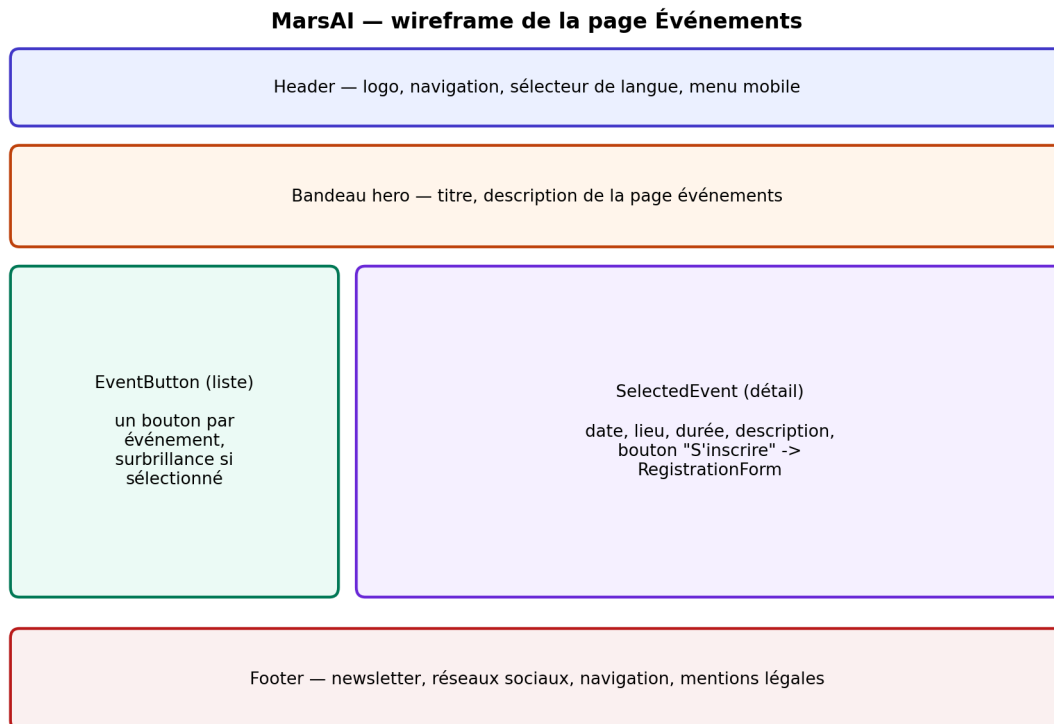


Figure — Wireframe de la page Événements (Header, EventButton, SelectedEvent, Footer).

J'ai aussi développé la page Galerie (Galery.jsx), qui affiche les contenus vidéo sélectionnés avec un système de recherche textuelle, de filtrage par catégorie, de tri (titre, année, pays) et de défilement infini implémenté avec un IntersectionObserver qui charge des lots supplémentaires de résultats au fur et à mesure du scroll, plutôt que de tout charger d'un coup.

Ma page Contact (contact.jsx) implémente une validation complète côté client (champs obligatoires, format d'email par expression régulière, longueur minimale du message) avant l'envoi vers /api/contact, avec des messages d'erreur affichés à la fois sous chaque champ et via un système de notification flash global (useFlash) que je consomme depuis un contexte React partagé sur l'ensemble du site.

J'ai construit la page FAQ (Faq.jsx) sous forme d'accordéon : les questions sont regroupées par section thématique (soumission, sélection, événement, droits, technique), et je gère l'ouverture d'une seule réponse à la fois avec un état local (expandedId).

Enfin, j'ai réalisé la page CguCgv.jsx qui présente les Conditions Générales d'Utilisation et de Vente, avec un léger effet de parallaxe calculé à partir de la position de scroll de la page.

Ce que je retiens de cette expérience de groupe

Ce projet en équipe m'a confronté à un contexte différent de mes projets personnels : je devais composer avec du code déjà écrit par d'autres membres de l'équipe (le contexte `SettingsContext`, le contexte `FlashContext`, la configuration `i18n`), respecter des conventions de nommage et de structure de dossiers déjà en place, et livrer des fonctionnalités testables par le client chaque semaine plutôt que d'itérer librement à mon propre rythme.

Je considère MarsAI comme complémentaire à mes deux autres projets, réalisés seul : il m'a permis de démontrer un travail d'équipe en méthode Agile et une conception collaborative des données, quand Astral Database et le Cahier de recettes collaboratif me permettent de démontrer une maîtrise complète, de la conception à la mise en œuvre, sur un périmètre que je maîtrise entièrement seul.

Sécurité et bonnes pratiques — MarsAI

Prévention des failles XSS

Sur mes composants frontend de MarsAI (`Header`, `Footer`, `Event_page`, `Faq`, `contact`, `CguCgv`, `Galery...`), mon risque XSS est structurellement limité par le fonctionnement de JSX, qui échappe automatiquement le contenu que j'insère via les expressions `{...}`.

Je n'utilise dans aucun de mes composants la prop `dangerouslySetInnerHTML`, y compris dans mon composant `Footer.jsx` qui affiche pourtant du contenu éditable par le client via un composant `Editable` partagé sur le projet : ce contenu passe par le rendu JSX standard et non par une insertion HTML brute, ce qui limite le risque même sur du texte modifiable dynamiquement par une personne non technique.

Gestion des secrets

Sur ma partie frontend de MarsAI, l'URL de l'API backend est lue depuis une variable d'environnement (`import.meta.env.VITE_API_URL`) plutôt que codée en dur, ce qui va dans le sens de cette bonne pratique, même si je n'ai pas la visibilité complète sur la gestion des secrets côté back-end de ce projet, géré par d'autres membres de l'équipe.

Chapitre 4 — Compétences transversales et conclusion

Pour finir ce dossier, je rassemble ici les compétences et pratiques qui traversent mes trois réalisations plutôt que de me rattacher à une seule d'entre elles : le respect du RGPD, ma capacité à communiquer techniquement en français et en anglais, ma démarche de résolution de problème et ma veille technique.

RGPD et données personnelles

Le référentiel me demande de mettre en place, en fonction du projet, les mentions légales liées au RGPD.

Je n'ai encore intégré, sur aucun de mes trois projets, de politique de confidentialité, ni de mécanisme de suppression ou d'anonymisation des données personnelles d'un compte utilisateur — mon modèle `User.js` du Cahier de recettes collaboratif ne prévoit par exemple pas de route de suppression de compte. C'est un point que je garde en tête pour la suite.

Communiquer en français et en anglais

La documentation technique que j'ai produite sur mon Cahier de recettes collaboratif, avec mes annotations Swagger consultables sur `/api-docs`, illustre ma capacité à communiquer techniquement à destination d'autres développeurs.

Je l'ai rédigée en français jusqu'à présent ; je prévois d'en traduire une partie en anglais, même partiellement, pour démontrer plus explicitement le niveau B1 de compréhension et d'expression écrite en anglais que le référentiel attend de moi — une action simple que je peux réaliser avant mon examen.

Mettre en œuvre une démarche de résolution de problème

Plusieurs choix que j'ai faits dans ce dossier illustrent, je crois, une démarche structurée face à une contrainte technique plutôt qu'une correction improvisée : j'ai géré les deux formats de clé `maxLevel` / `max_level` dans `useSkillTree.js` parce que l'API Mihomo n'est pas parfaitement stable dans le nommage de ses champs selon les versions ; j'ai mis en place un repli en cascade sur une disposition

par défaut dans `getLayout` lorsque le nom de la voie d'un personnage n'est reconnu par aucune de mes clés de correspondance ; j'ai construit une double requête SQL (préfixe / contient) dans `autocomplete.php` pour obtenir un tri de pertinence sans complexifier ma requête.

Dans chaque cas, j'ai isolé le problème, formulé une hypothèse de cause, puis mis en place une solution défensive plutôt que de simplement faire disparaître le symptôme.

Apprendre en continu

Sur Astral Database, j'ai volontairement mobilisé des outils relativement récents (Vite 8 avec le bundler Rolldown, React Compiler encore en phase d'adoption progressive dans l'écosystème React 19), ce qui m'a demandé une veille technique active pendant mon développement pour suivre une documentation encore mouvante.

De la même façon, j'ai dû explorer et tester de façon itérative le format de réponse de l'API Mihomo, qui n'est pas officiellement documentée par un fournisseur mais par la communauté du jeu — j'ai laissé volontairement des traces de ce travail de découverte dans mon code, par exemple les lignes `console.log` de suivi de chargement du cache dans `lightConeCache.js`, plutôt que de les effacer une fois le problème résolu.